Step 4 — An Analysis Agent That Turns Raw Research Into Organized Findings — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: raw notes are a flat list; a good article needs structure. The Analysis agent groups the notes into themes — **without losing any fact**. It's the first step that feeds one built agent's *real* output into another, so an interface mismatch between Research and Analysis shows up now, before a Supervisor hides it. **If not:** the Writer would get an unordered list, or silently lose a fact that didn't fit a theme.

After this step the project runs: research → analysis → write → review. Files not shown here don't change.

Solution — themes, every fact kept, checked on Research's real output

Story — `prompts/analysis.txt`: 2-4 themes, keep **every** fact and its tag, and an "Other" theme for facts that don't fit. **If not:** the article would quietly lose facts between Research and Writer.

`src/contentforge/prompts/analysis.txt`

```
You are the Analyst on a content team. Organize the research notes below
into 2 to 4 themes. Under each theme, list its facts as bullet points.
Keep EVERY fact and its [source] tag - do not drop a fact just because it
does not fit a theme neatly; put it under "Other" instead.

Research notes:
{notes}
```

Story — `agents/analysis_agent.py`: a plain chain, no tool — organizing is thinking, not looking up. **If not:** you'd give it a tool it never needs.

```
# src/contentforge/agents/analysis_agent.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from contentforge.models.llm import get_model
```

```
from contentforge.utils.prompts import load_prompt

def run_analysis(notes: str) -> str:
    # why: a plain chain, no tool — organizing is thinking, not looking up
    prompt = ChatPromptTemplate.from_template(load_prompt("analysis"))
    chain = prompt | get_model() | StrOutputParser()
    return chain.invoke({"notes": notes})
```

Changed from Step 3: `main.py` runs Analysis between Research and the writing loop.

```
# src/contentforge/main.py
import sys

from contentforge.agents.analysis_agent import run_analysis
from contentforge.agents.research_agent import run_research
from contentforge.agents.reviewer_agent import PASS_SCORE, run_reviewer
from contentforge.agents.writer_agent import run_writer
from contentforge.config import load_config
from contentforge.exceptions import MissingConfigError
from contentforge.utils.logger import get_logger

# why: the hard limit on Writer <-> Reviewer rounds — never loop forever
MAX_REVISIONS = 3

def write_with_review(findings: str) -> dict:
    # why: returns a dict with the SAME keys the Step 5 team state uses,
    # so print_report() never has to change its input
    logger = get_logger(__name__)
    result = {
        "draft": "",
        "accepted": False,
        "feedback_history": [],
        "revision_count": 0,
    }
    feedback = ""
    for round_number in range(1, MAX_REVISIONS + 1):
        result["draft"] = run_writer(findings, feedback)
        verdict = run_reviewer(findings, result["draft"])
        logger.info("round %s: score %s", round_number, verdict.score)
        if verdict.score >= PASS_SCORE:
            result["accepted"] = True
            return result
        # how: the Reviewer's feedback goes into the Writer's next round
        feedback = verdict.feedback
        result["feedback_history"].append(feedback)
        result["revision_count"] = result["revision_count"] + 1
```

```
    return result

def print_report(result: dict) -> None:
    if result["accepted"]:
        print("\nFinal article:\n")
        print(result["draft"])
    else:
        # why: never pass a rejected draft off as finished
        attempts = result["revision_count"]
        print(f"\nCouldn't agree after {attempts} revisions. Feedback:")
        number = 1
        for feedback in result["feedback_history"]:
            print(f"  round {number}: {feedback}")
            number = number + 1
        print("\nLast draft (NOT approved):\n")
        print(result["draft"])

def main() -> None:
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    topic = input("Topic: ").strip()
    notes = run_research(topic)
    if notes == "":
        print("No article: nothing is known about this topic yet.")
        return
    # new in Step 4: Analysis organizes the notes before the Writer
    findings = run_analysis(notes)
    print(f"Findings:\n{findings}\n")
    result = write_with_review(findings)
    print_report(result)

if __name__ == "__main__":
    main()
```

Changed from Step 3: the integration test adds case 4 — Research's **real** output goes in, and the `[source]` tags are counted before and after. Same count = no fact dropped.

```
# tests/integration/test_live_agents.py
# Runs the agents against the real OpenAI API.
# Run from the repo root: python tests/integration/test_live_agents.py
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from contentforge.agents.analysis_agent import run_analysis
```

```
from contentforge.agents.research_agent import run_research
from contentforge.agents.reviewer_agent import PASS_SCORE, run_reviewer
from contentforge.config import load_config

FINDINGS = (
    "Range:\n- A battery gives 40 to 100 km per charge. [battery-review]\n"
    "Rules:\n- EU e-bike motors are limited to 25 km/h. [eu-rules]\n"
)

def test_reviewer_rejects_weak_draft() -> None:
    # case 1: a weak draft must fail — a Reviewer that passes everything
    # would never make the revision loop run
    verdict = run_reviewer(FINDINGS, "E-bikes are good.")
    passed = verdict.score >= PASS_SCORE
    print(f"case 1 (weak draft)     -> score {verdict.score}, "
          f"passed={passed}")

def test_research() -> None:
    # case 2: a known topic gives notes that keep the [source] tags
    notes = run_research("electric bikes")
    print(f"case 2 (known topic)    -> {notes.count('[')} sourced facts")

    # case 3: an unknown topic gives nothing — never invented notes
    notes = run_research("deep sea mining")
    print(f"case 3 (unknown topic)  -> {notes!r}")

def test_analysis_keeps_every_fact() -> None:
    # case 4: organizing must not drop a fact — compare the [source]
    # tags before and after, on Research's REAL output
    notes = run_research("electric bikes")
    findings = run_analysis(notes)
    print(f"case 4 (facts kept)     -> {notes.count('[')} in, "
          f"{findings.count('[')} out")

if __name__ == "__main__":
    load_config()
    test_reviewer_rejects_weak_draft()
    test_research()
    test_analysis_keeps_every_fact()
```

Run it:

```
python tests/integration/test_live_agents.py
python -m contentforge.main
```

```
case 4 (facts kept)     -> 5 in, 5 out
```